

11- Bilgisayar Altyapısında Performans Modellemesi

Unat Teksen

20 Nisan

1 Neden Performans Önemlidir?

- **Performans Tanımı:** Bir bilgisayar sisteminin toplam etkinliğidir; throughput (iş hacmi), bireysel yanıt süresi ve erişilebilirlik açısından ölçülür. Daha basit bir ifadeyle: harcanan zaman ve kaynaklara kıyasla ne kadar yararlı iş yapılabildiğidir.

1.1 Neden Bu Kadar Önemli? (Gerçek Dünya Örnekleri)

- **Amazon (2006):** Sayfa yüklenme süresindeki her **100ms** gecikme → **%1 satış düşüşü** → 2006'da 107 milyon dolar kayıp (Bugün bu rakam 3.8 milyar dolar olurdu).
- **Google (Aynı dönem):** Arama sayfası oluşturma süresindeki **0.5 saniyelik** gecikme → **%20 trafik kaybı**.
- **Finansal Sektör:** Bir broker'ın sistemi rakiplerinden 5ms yavaş olursa → **millisaniye başına 4 milyon dolar kayıp**.

Ana Ders

Performans sadece teknik bir konu değil, doğrudan iş sonuçlarını (gelir, trafik, itibar) etkileyen kritik bir faktördür.

2 Performans Nasıl Değerlendirilir?

Bir sistemi performans açısından değerlendirmek için üç temel yaklaşım vardır:

2.1 Yaklaşım 1: Sezgi ve Trend Ekstrapolasyonu

- Deneyime dayalı tahmin.
- **Avantaj:** Hızlı ve esnek.
- **Dezavantaj:** Düşük doğruluk — bu kalitede sezgiye sahip insanlar nadirdir.

2.2 Yaklaşım 2: Deneysel Değerlendirme (Measurement-Based)

Gerçek sistemde fiziksel ölçüm yapılır. Üç alt yöntemi vardır:

- **Avantaj:** Mükemmel doğruluk (Gerçek değerleri verir).

Yöntem	Açıklama
Direct Measurement	Çalışan sistemi doğrudan ölçmek.
Benchmarking	Standart testlerle farklı sistemleri karşılaştırmak.
Prototype	Küçük ölçekli prototip üzerinde test yapmak.

- **Dezavantaj:** Çok zahmetli, pahalı ve **esnek değil**. Bir senaryo için doğru sonuç verir ama genelleme yapılamaz. "What-if" (ya şöyle olursa) analizi için esneklik çok düşüktür.

2.3 Yaklaşım 3: Model Tabanlı Değerlendirme (Model-Based) ★

Sistemin soyutlanmış bir **modeli** üzerinden çalışılır.

- **Model Nedir? (Lazowska'nın Tanımı):** "Sistemin davranışı açısından özsel olan yönleri, sistem detaylarından damıtan bir temsil."
- **Modelin Özellikleri:** Gerçek sistemden **daha basittir**, **temel karakteristikleri** yakalar ve **tahmin yapmak** için değerlendirilir.
- **Analoji:** Bir bina için maket (maquette) yapmak — gerçek bina değildir ama temel özellikleri (tasarım, hacim) yansıtır.

Model Tabanlı Alt Yöntemler:

Yöntem	Açıklama	Avantaj	Dezavantaj
Analitik / Sayısal	Matematiksel, olasılık teorisi.	En verimli, en kesin.	Sadece sınırlı durumlarda uygulanabilir.
Simülasyon	Modelin izlerini (trace) taklit eder.	En genel yöntem.	Nadir olaylar için az doğru; yüksek doğruluk için uzun süre gerekir.
Hibrit	İkisini birleştirir.	Dengeli.	-

Ana Ders

Bu derste kullanılacak temel yöntem: **Analitik ve sayısal yöntem** → **Kuyruk Ağı Modelleri (Queueing Networks)**.

2.4 Model Tanımlama Süreci

Model şu adımlarla oluşturulur ve değerlendirilir:

Mevcut Sistem / Sistem Tasarımı → **Hedefi Belirle** → **Modeli Oluştur** → **Modeli Parametrize Et** → **Modeli Değerlendir** → **Performans İndeksleri (Sonuçlar)**

Exam Focus (Sınav Odak Noktası)

- Measurement-based vs Model-based farkları. Measurement'ın "what-if" analizi için düşük esnekliğe sahip olduğunu bilin.
- Model tabanlı yöntemin alt türleri: Analitik, Simülasyon, Hibrit.

3 Kuyruk Ağları (Queueing Networks)

3.1 Kuyruk Teorisi Nedir?

- **Tanım:** Çok sayıda iş, kıt kaynaklar ve bu yüzden oluşan kuyruklar ve gecikmeler olduğunda ne olduğunu açıklayan matematiksel teoridir.
- **Bilgisayarda Kullanımı:** CPU (zamanlayıcı kuyruğu), Disk (okuma/yazma sırası), Router (paket kuyruğu), Veritabanı (kilit sıraları).

3.2 Kuyruk Ağı (Queueing Network) Nedir?

- **Tanım:** Bilgisayar sisteminin **bir kuyruklar ağı** olarak temsil edildiği özel bir performans modelleme yaklaşımıdır.
- **Bileşenleri:**
 1. **Service Centers (Servis Merkezleri):** Sistem kaynaklarını temsil eder (CPU, disk, ağ).
 2. **Customers/Jobs (Müşteriler/İşler):** Kullanıcıları veya işlemleri temsil eder.
- **Başarının Sırrı:** Bir sistemin düşük seviyeli detayları, yüksek seviyeli performans karakteristikleri açısından büyük ölçüde önemsizdir. (CPU'nun transistör sayısı değil, ortalama servis süresi önemlidir).

4 Tek Kuyruk (Single Queue)

Kuyruk ağını anlamak için önce tek bir kuyruğu anlamamız gerekir.

Temel Senaryo:

[Arrivals] → [Buffer/Queue] → [Server] → [Departures]

- **İşleyiş:**
 - Müşteriler (işler) sisteme gelir (Arrivals).
 - Server meşgulse müşteri **buffer'da** (tamponda) bekler.
 - Server boşaldığında buffer'dan sıradaki müşteri seçilir (Servis disiplinine göre).
 - Servis tamamlandıca müşteri sistemi terk eder (Departures).
- **Analoji:** Banka gişesi. Gişeler doluysa sıraya girersin. Boşalınca sıradaki çağrılır.

5 Servis Merkezini Karakterize Eden Dört Temel Özellik

Bir kuyruk modeli dört temel özellik ile tanımlanır:

5.1 Özellik 1: Arrival (Varış)

- **Tanım:** Sisteme giren işleri temsil eder — ne kadar hızlı, ne sıklıkla ve hangi tür işlerin geldiğini belirtir.
- **Temel Metrik:** λ (**Lambda**) — **Ortalama varış hızı**. Birim: req/s (istek/saniye).
- **Varış Kaynakları:**
 1. **Dış kaynak:** Dış Dünyadan $\rightarrow$ Kuyruk-Server'a.
 2. **Başka kuyruk:** Kuyruk1-Server1 $\rightarrow$ Kuyruk2-Server2'ye.
 3. **Aynı kuyruk (Loop-back):** Kuyruk-Server çıkışından $\rightarrow$ tekrar kendi girişine.
- **Önemli:** λ bir "ortalama" değerdir. ($\lambda = 100$ req/s $\rightarrow$ Saniyede ortalama 100 istek gelir).

5.2 Özellik 2: Service (Servis)

- **Tanım:** Bir işin işlenirken geçirdiği zamanı temsil eder.
- **Servis Süresi:** Bir sunucunun müşteriye hizmet için harcadığı süredir.
- **Temel Metrikler:**
 - **Ortalama Servis Süresi:** $1/\mu$
 - **Maksimum Servis Hızı:** μ (Mu). Birim: req/s.
- **Örnek:** Web sunucusu her isteği ortalama 2ms'de işliyorsa:
 - Ortalama servis süresi $= 1/\mu = 2\text{ms} = 0.002\text{s}$.
 - Maksimum servis hızı $= \mu = 1/0.002 = 500$ req/s.
- **Dikkat:** μ , maksimum *teorik* hızdır. Gerçekte sunucu her zaman meşgul değildir.

5.3 Özellik 3: Number of Servers (Sunucu Sayısı)

Bir servis merkezinde kaç sunucu olabileceğine dair üç seçenek vardır:

1. **Tek Sunucu (Single Server):**
 - Aynı anda sadece 1 müşteri servis alabilir. Diğerleri buffer'da bekler.
 - Sunucu meşgulken gelen müşteri **mutlaka sıraya girer**.
2. **Çoklu Sunucu (Multiple Servers - c adet):**
 - **c adet** sunucu paralel çalışır.
 - Müşteri sayısı $\leq c$ ise: Bekleme yok, hepsi direkt servis alır.
 - Müşteri sayısı $> c$ ise: Fazla müşteriler buffer'da bekler.
3. **Sonsuz Sunucu (Infinite Server):**

- Her zaman en az müşteri kadar sunucu vardır. **Hiç bekleme (kuyruk) olmaz.** Buffer yoktur.
- Gerçek dünyada nadirdir. Genellikle kullanıcının ”**düşünme süresini**” (**think time**) modellemek için kullanılır.

5.4 Özellik 4: Queue / Buffer Capacity (Kuyruk Kapasitesi)

- **Tanım:** Buffer, servis bekleyen müşterilerin beklediği alandır.
- **Buffer dolduğunda iki seçenek vardır:**
 1. **Arrivals Suspended (Varışlar Askıya Alınır):** Sistem dolu gerçeği geri bildirilir. Bir müşteri ayrılana kadar yeni müşteri alınmaz.
 2. **Customers are Lost (Müşteriler Kaybolur):** Yeni gelen müşteriler **geri çevrilir** (Örn: İnternette paket kaybı/drop).
- **Sonsuz Buffer:** Kapasite, sistem davranışını hiç etkilemeyecek kadar büyükse teorik olarak sonsuz kabul edilir. (Genel varsayım budur).
- **Örnek (Slide 39):** Wireless gateway. $\lambda = 125$ paket/s. Servis süresi = 2ms ($\mu = 500$ paket/s). Kapasite = 13 yer. Buffer dolunca paket kaybolur. (Soru: Milyonda 1 kayıp için bu kapasite yeterli mi? - Modelleme ile çözülür).

5.5 Özellik 4 (Devamı): Service Discipline (Servis Disiplini)

- **Tanım:** Sunucu boşaldığında buffer'daki hangi müşterinin seçileceğini belirleyen kuraldır.

Kısaltma	Açılım (Türkçe)	Açıklama / Analoji
FCFS / FIFO	First-Come-First-Serve (İlk Gelen İlk Çıkar)	En yaygın. (Normal market kuyruğu).
LCFS / LIFO	Last-Come-First-Serve (Son Gelen İlk Çıkar)	Yığın/Stack mantığı. (Asansörden çıkış).
RSS	Random Selection for Service (Rastgele Seçim)	Şapkadan kura çekmek gibi.
PRI	Priority (Öncelik)	Yüksek öncelikli olan önce alınır. (Acil servis triajı).

Exam Focus (Sınav Odak Noktası)

- Dört temel özelliği ezberleyin (Arrival, Service, Number of Servers, Buffer/Discipline).
- λ (Ortalama varış hızı) ve μ (Maks servis hızı) anlamlarını ve birimlerini (**req/s**) bilin.
- FCFS, LCFS, RSS, PRI farklarını açıklayın.
- Single, Multiple ve Infinite server farkını bilin (Infinite = bekleme yok, think time).
- Sonlu (finite) buffer durumundaki iki senaryoyu bilin (Askıya Alma vs. Paket Kaybı).

6 Özellik 5: Population (Nüfus)

6.1 Population Nedir?

- **Tanım:** Sisteme iş gönderebilecek müşterilerin toplamıdır. Temel özellik büyüklüğüdür (size).

İki Tür Population:

1. Finite Population (Sonlu Nüfus):

- Maksimum N müşteri vardır.
- Tüm N müşteri zaten sistemdeyse $\rightarrow$ yeni varış **sıfır** olur.
- Varış hızı λ , sistemdeki müşteri sayısından etkilenir.
- *Analoji:* Bir şirkette 50 bilgisayar var. Hepsi güncelleme sunucusuna bağlarsa, yeni istek gönderecek bilgisayar kalmaz.

2. Infinite Population (Sonsuz Nüfus):

- Nüfus o kadar büyüktür ki varış sürecini hiç etkilemez.
- λ sabittir, sistemdeki müşteri sayısından bağımsızdır. Teorik analizde en çok kullanılan varsayım budur.
- *Analoji:* Büyük bir e-ticaret sitesine gelen kullanıcılar. Birinin istek göndermesi diğerlerini etkilemez.

6.2 Population Classes (Nüfus Sınıfları)

- İdeal durumda tüm müşteriler birbirinden ayırt edilemez (homojen).
- Farklılık varsa nüfus **sınıflara (classes)** ayrılır. Aynı sınıftakiler aynı davranışı sergiler. Farklı sınıflar; varış hızı, servis süresi vb. özelliklerde farklılık gösterir.
- Farklı sınıfları belirlemeye **workload characterization (iş yükü karakterizasyonu)** denir.
- *Örnek:* Web sunucusunda mobil kullanıcılar (küçük/hızlı istekler) vs. masaüstü kullanıcılar (büyük/yavaş istekler) $\rightarrow$ 2 farklı sınıf.

Exam Focus (Sınav Odak Noktası)

- Finite vs Infinite population farkını bilin.
- Finite population'da λ 'nın neden değişkenlik gösterdiğini açıklayabilm (Müşteri sayısı limitlidir).
- Nüfus sınıflarının neden gerekli olduğunu (farklı davranışlı iş yüklerini modellemek) bilin.

7 Özellik 6: Routing (Yönlendirme)

7.1 Routing Nedir?

- **Tanım:** Bir iş, servisini tamamladıktan sonra birden fazla olası yolu varsa, bir sonraki hedefin nasıl seçileceğini tanımlayan politikadır.
- **Önemli Not:** Routing, yalnızca bir işin **birden fazla olası hedefi** olduğu noktalarda gereklidir. Tek yol varsa routing gerekmez.

7.2 Routing Algoritmaları

1. Probabilistic Routing (Olasılıksal Yönlendirme):

- Her yola bir olasılık atanır. İş bu olasılıkla hedefe gider.
- p değerlerinin **toplamı** = 1 olmalıdır. (Örn: %20 A'ya, %50 B'ye, %30 C'ye).

2. Round Robin Routing:

- Hedef, tüm olası çıkışlar arasında sırayla döner (İş 1 $\rightarrow$ A, İş 2 $\rightarrow$ B, İş 3 $\rightarrow$ C, İş 4 $\rightarrow$ tekrar A).

3. Join the Shortest Queue (JSQ - En Kısa Kuyruğa Katıl):

- İş, olası hedeflerin kuyruk uzunluklarını sorgular ve **en az iş bekleyen** sunucuya gider. Süpermarkette en kısa kasaya geçmek gibidir.

Exam Focus (Sınav Odak Noktası)

- Üç routing algoritmasını tanımlayabilin.
- Routing'ın sadece birden fazla alternatif yol olduğunda gerekli olduğunu bilin.
- JSQ'da en kısa kuyruğun seçileceğini hesaplayabilin.

8 Kuyruk Ağı Tipleri: Open, Closed, Mixed

8.1 Open Network (Açık Ağ)

- **Tanım:** Müşterilerin dış ortamdan gelip dış ortama gittiği ağlardır. Dışarıdan *arrivals* ve dışarıya *departures* vardır.
- **Karakteristikleri:** Dışarıdan sabit bir λ ile iş gelir. İşler bitince sistemi terk eder. **Toplam iş sayısı değişkendir.**
- **Örnek:** 3 katlı web uygulaması (İnternet $\rightarrow$ Web Server $\rightarrow$ App Server $\rightarrow$ DBMS $\rightarrow$ Çıkış).

Tandem Network (Seri Ağ): Özel bir açık ağ türüdür. İşler **sırayla** her servis merkezinden geçer (Giriş $\rightarrow$ A $\rightarrow$ B $\rightarrow$ Çıkış). Fabrika üretim hattı gibidir.

8.2 Closed Network (Kapalı Ağ)

- **Tanım:** Sabit sayıda müşterinin (N) sistem içinde sürekli dolaştığı ağlardır. Dışarıdan giriş veya dışarıya çıkış **yoktur**.
- **Karakteristikleri:** N sabittir. İşler bitince sisteme geri döner.
- **Think Time (Düşünme Süresi):** Kapalı ağlarda müşteriler servisi aldıktan sonra hemen yeni istek göndermez, "düşünürler". Bu süre **Infinite Server (Delay Center)** ile modellenir.
- **Örnek:** Şirket içi sistem. Sınırlı sayıda laptop, iç ağ üzerinden sunuculara sürekli döngü halinde erişir.

8.3 Mixed Network (Karma Ağ)

Hem açık hem kapalı davranış gösteren müşteri sınıflarının bir arada bulunduğu ağdır.

Exam Focus (Sınav Odak Noktası)

- Open vs Closed network farkını net bilin.
- Open: Dışarıdan geliş/gidiş var, iş sayısı değişken.
- Closed: Dışarıya yok, iş sayısı (N) sabit, *Think time* var.
- Think time'in Infinite Server ile modellendiğini bilin.
- Tandem network'ün sıralı bir açık ağ olduğunu bilin.

9 Model Detay Seviyesi (Level of Detail)

Kuyruk ağı modeli oluştururken ne kadar detay ekleneceği kritik bir karardır.

- **Düşük:** Tek bir kuyruk (Tüm Web Server).
- **Orta:** CPU ve Disk ayrı.
- **Yüksek:** RAM, Cache, Disk hepsi ayrı.
- **Kural: "Proceed top-down" (Yukarıdan aşağıya ilerle).** Önce basit başla, gerektiğinde detaylandır.
- **Neden?:** Detay arttıkça parametrize etmek (gerçek değerleri atamak) zorlaşır. Karmaşıklık düşük tutun. Sadece analiz hedefine ulaşmak için gereken detayı ekleyin. Gereksiz detay = hata riski artışı.

10 Kuyruk Ağının Tam Bileşenleri Özet Tablosu

11 Genel Master Özet — Tüm Konular (Karar Ağacı)

- **Model Tipi Seçimi:**
 - Dışarıdan müşteri geliyor mu? → **EVET** → **Open Network** (Sıralıysa → Tandem).

Bileşen	Açıklama	Temel Metrik / Tür
Arrival	İşlerin sisteme girişi	λ (varış hızı, req/s)
Service	İşlerin işlenmesi	μ (servis hızı), $1/\mu$ (servis süresi)
Queue/Buffer	Bekleme alanı	Kapasite (finite/infinite)
Population	Müşteri havuzu	N (finite) veya ∞ (infinite)
Routing	Servis sonrası yönlendirme	Probabilistic, Round Robin, JSQ
Service Discipline	Kuyruktan seçim kuralı	FCFS, LCFS, RSS, PRI
Servers	Paralel sunucu sayısı	c (single/multi/infinite)
Network Type	Ağ yapısı	Open, Closed, Mixed

- Dışarıdan müşteri geliyor mu? → **HAYIR** → **Closed Network** (Think time var mı? → Infinite Server ekle).

- **Routing Seçimi:**

- Bir sonraki hedef belli mi? → **EVET** → Routing **gerekmez**.
- Belli değilse → Olasılıklı (Probabilistic), Sırayla (Round Robin), En kısa kuyruk (JSQ).

Ana Ders

Büyük Resim: Bu derste öğrenilen **Queueing Network Theory**, bilgisayar altyapılarında (Data Center, Storage, Virtualization vb.) **Performans (Throughput & Response Time) analizi** için kullanılır.